



MATLAB Numerical Methods Quiz

Preparation Guide

Quiz Date: October 20, 2025

Topics: Root Finding Methods (Bisection, Newton-Raphson, Secant, Regula Falsi, Fixed Point, Modified Newton)



UNIVERSAL MATLAB TEMPLATE FOR ALL ROOT-FINDING METHODS

USE THIS EXACT PATTERN - Change only the formulas and method names!

```
%=====
% METHOD NAME: [Method Name Here]
%=====

clc
clear
close all

%=====
% STEP 1: DEFINE FUNCTION AND DERIVATIVES (If needed)
%=====

f = @(x) [FUNCTION HERE];           % Main function

% Uncomment derivative if method requires it
% df = @(x) [DERIVATIVE HERE];      % First derivative (Newton, Modified Newton)
% d2f = @(x) [2ND DERIVATIVE HERE]; % Second derivative (Modified Newton)

%=====
% STEP 2: INPUT INITIAL GUESS(ES)
%=====

% For Bisection/Regula Falsi (bracket methods):
% a = input('Enter value of a: ');
% b = input('Enter value of b: ');

% For Newton/Secant/Fixed Point (open methods):
x0 = input('Enter initial guess x0: ');

%=====
% STEP 3: VALIDATE INPUT (For bracket methods)
%=====

% Uncomment for bracket methods:
% while f(a) * f(b) > 0
%     disp('f(a) and f(b) must have opposite signs!');
%     a = input('Enter new value of a: ');
%     b = input('Enter new value of b: ');
% end

%=====
% STEP 4: SET PARAMETERS
%=====
```

```

tol = 1e-6;          % Tolerance (convergence criterion)
maxIter = 50;        % Maximum iterations
i = 0;              % Iteration counter
err = inf;           % Initialize error

%-----
% STEP 5: PRINT HEADER
%-----

fprintf('\n%s\n',
'=====');
fprintf('%-5s %-15s %-15s %-15s %-15s\n', ...
'Iter', 'x_value', 'f(x)', 'Error(%)', 'Other');
fprintf('%s\n',
'=====');

%-----
% STEP 6: ITERATION LOOP
%-----

while (err > tol) && (i < maxIter)
    i = i + 1;

    % *** PASTE YOUR METHOD-SPECIFIC FORMULA HERE ***
    % Different for each method!

    % Compute error
    if i == 1
        err = inf; % First iteration has no previous value
    else
        err = abs((x_new - x_old) / x_new) * 100; % Relative error
    end

    % Print iteration results
    fprintf('%-5d %-15.8f %-15.8f %-15.6f\n', i, x_new, f(x_new), err);

    % Update for next iteration
    x_old = x_new;
    % x0 = x_new; (uncomment based on method)
end

%-----
% STEP 7: DISPLAY FINAL RESULT
%-----

fprintf('%s\n',
'=====');
fprintf('\n✅ Approximate root = %.8f\n', x_new);
fprintf('✅ Found in %d iterations\n', i);
fprintf('✅ Tolerance = %.1e\n', tol);
fprintf('✅ Final error = %.6e\n\n', err);

```

METHOD-SPECIFIC FORMULAS (The ONLY thing that changes!)

1. BISECTION METHOD

- **Category:** Closed/Bracket method
- **Requires:** Function only
- **Formula in Loop:**

```
Xm = (a + b) / 2;                                % Midpoint
if f(a) * f(Xm) < 0
    b = Xm;                                       % Root in [a, Xm]
else
    a = Xm;                                       % Root in [Xm, b]
end
x_new = Xm;
```

2. NEWTON-RAPHSON METHOD

- **Category:** Open method
- **Requires:** Function + First derivative
- **Formula in Loop:**

```
f0 = f(x0);
f1 = df(x0);
x_new = x0 - f0 / f1;                            % Newton's formula
x0 = x_new;
```

3. SECANT METHOD

- **Category:** Open method
- **Requires:** Function only (no derivative!)
- **Requires:** TWO initial guesses (x0, x1)
- **Formula in Loop:**

```
x2 = x1 - f(x1) * (x1 - x0) / (f(x1) - f(x0));
x0 = x1;
x1 = x2;
x_new = x2;
```

4. REGULA FALSI METHOD (False Position)

- **Category:** Closed/Bracket method
- **Requires:** Function only
- **Formula in Loop:**

```
x_new = (a*f(b) - b*f(a)) / (f(b) - f(a)); % Linear interpolation  
  
if f(a) * f(x_new) < 0  
    b = x_new; % Root in [a, x_new]  
else  
    a = x_new; % Root in [x_new, b]  
end
```

5. FIXED POINT ITERATION

- **Category:** Open method
- **Requires:** Rearranged function $g(x)$ where $x = g(x)$
- **Condition:** $|g'(x)| < 1$ at root for convergence
- **Formula in Loop:**

```
x_new = g(x0); % Simple iteration formula  
  
x0 = x_new;
```

6. MODIFIED NEWTON-RAPHSON METHOD

- **Category:** Open method
- **Requires:** Function + First & Second derivatives
- **Used for:** Multiple roots (when $f(x) = 0$ AND $f'(x) = 0$)
- **Formula in Loop (Variant 1):**

```
f0 = f(x0);  
f1 = df(x0);  
f2 = d2f(x0);  
  
x_new = x0 - (f0 * f1) / (f1^2 - f0 * f2);  
  
x0 = x_new;
```



QUICK REFERENCE TABLE

Method	Type	Input	Convergence	Best For
Bisection	Closed	$f(x)$, $[a,b]$	Slow, Guaranteed	Bracketing
Newton-Raphson	Open	$f(x)$, $f'(x)$, x_0	Fast, Quadratic	Simple roots
Secant	Open	$f(x)$, x_0 , x_1	Fast	No derivative
Regula Falsi	Closed	$f(x)$, $[a,b]$	Medium	Bracketing
Fixed Point	Open	$g(x)$, x_0	Variable	Rearranged form
Modified NR	Open	$f(x)$, $f'(x)$, $f''(x)$, x_0	Very Fast	Multiple roots



KEY CONCEPTS TO REMEMBER

Error Calculations

```
% Absolute Error
err_abs = abs(x_new - x_old);

% Relative Error (percentage)
err_rel = abs((x_new - x_old) / x_new) * 100;
```

Stopping Criteria

```
% Stop when:
while (err > tol) && (i < maxIter)
    % - Error is less than tolerance, OR
    % - Maximum iterations reached
end
```

Convergence Checks

```
% Bracket methods (Bisection, Regula Falsi):
if f(a) * f(b) > 0
    % Same sign = no root in interval!
end

% Fixed Point:
if abs(dg_dx) >= 1
    % Won't converge!
end
```



COMMON MISTAKES TO AVOID

1. **✗ Forget `clc; clear; close all;`** → Always clean workspace
2. **✗ Wrong error formula** → Use relative error: $\text{abs}((\text{new} - \text{old})/\text{new}) * 100$
3. **✗ Check bracket validity** → $f(a) \cdot f(b) < 0$ MUST be true
4. **✗ Update variables wrongly** → Track which variable gets updated

5. **✗ Print header before loop** → Makes table readable
 6. **✗ Forget to initialize err = inf** → So first iteration doesn't trigger early stop
 7. **✗ Wrong derivative** → Double-check your calculus!
-



STEP-BY-STEP WRITING STRATEGY FOR QUIZ

When writing by hand, follow this order:

1. **Write header section** (clc, clear, close all)
 2. **Define function** (f = @(x) ...)
 3. **Get input** (x0 = input(...))
 4. **Set parameters** (tol, maxIter, i=0, err=inf)
 5. **Print header** (fprintf for table)
 6. **Write while loop condition** (err > tol && i < maxIter)
 7. **Update iteration counter** (i = i + 1)
 8. **Write method formula** (THE KEY PART!)
 9. **Calculate error** (err = abs((new-old)/new)*100)
 10. **Print results** (fprintf)
 11. **Update variables** (x0 = x_new, etc)
 12. **Close loop and print final result**
-



YOUR ACTUAL CODE ANALYSIS

Your codes follow this structure:

- ✓ Proper initialization
- ✓ Good error calculations
- ✓ Clear output formatting
- ✓ Proper stopping criteria

Standardize by using the template above!



PRACTICE PROBLEMS

Bisection Method

Find root of

in interval [2, 3]

$$f(x) = x^3 - 2x - 5$$

Newton-Raphson

Find root of

$$f(x) = e^x - 5x$$

with $x_0 = 1$

Secant Method

Find root of

$$f(x) = \cos(x)$$

with $x_0 = 0, x_1 = 1$

Regula Falsi

Find root of

$$f(x) = x^3 - 2x - 5$$

in interval $[2, 3]$

Fixed Point

Rearrange

$$x^3 + x - 1 = 0$$

to

$$x = \frac{1}{x^2 + 1}$$

and iterate from $x_0 = 0.5$

✨ FINAL TIPS FOR QUIZ

1. **Use the template!** Write it down first
2. **Identify the method clearly** (what's being asked)
3. **Choose correct formula** from the table
4. **Show error calculation** step-by-step
5. **Keep formatting consistent** for easy grading
6. **Test your code mentally** before submitting
7. **Label all outputs** clearly

=====

QUICK SHEET PREP

Skip to content
Chat history

You said:

give in good form, so that i can easily copy it and paste it in word, don't change anythings "# 🎓
MATLAB Numerical Methods Quiz Preparation Guide

****Quiz Date:**** October 20, 2025

****Topics:**** Root Finding Methods (Bisection, Newton-Raphson, Secant, Regula Falsi, Fixed Point, Modified Newton)

📄 UNIVERSAL MATLAB TEMPLATE FOR ALL ROOT-FINDING METHODS

****USE THIS EXACT PATTERN - Change only the formulas and method names!****

```
matlab
%=====
% METHOD NAME: [Method Name Here]
%=====

clc
clear
close all

%=====
% STEP 1: DEFINE FUNCTION AND DERIVATIVES (If needed)
%=====

f = @(x) [FUNCTION HERE];           % Main function

% Uncomment derivative if method requires it
% df = @(x) [DERIVATIVE HERE];      % First derivative (Newton, Modified Newton)
% d2f = @(x) [2ND DERIVATIVE HERE]; % Second derivative (Modified Newton)

%=====
% STEP 2: INPUT INITIAL GUESS(ES)
%=====

% For Bisection/Regula Falsi (bracket methods):
% a = input('Enter value of a: ');
% b = input('Enter value of b: ');

% For Newton/Secant/Fixed Point (open methods):
x0 = input('Enter initial guess x0: ');

%=====
% STEP 3: VALIDATE INPUT (For bracket methods)
%=====

% Uncomment for bracket methods:
% while f(a) * f(b) > 0
%     disp('f(a) and f(b) must have opposite signs!');
%     a = input('Enter new value of a: ');
%     b = input('Enter new value of b: ');
% end

%=====
% STEP 4: SET PARAMETERS
%=====

tol = 1e-6;           % Tolerance (convergence criterion)
maxIter = 50;         % Maximum iterations
i = 0;               % Iteration counter
err = inf;            % Initialize error

%=====
% STEP 5: PRINT HEADER
%=====

fprintf('\n%s\n', '=====');
fprintf('%-5s %-15s %-15s %-15s %-15s\n', ...
    'Iter', 'x_value', 'f(x)', 'Error(%)', 'Other');
fprintf('%s\n', '=====');
```

```

%-----
% STEP 6: ITERATION LOOP
%-----

while (err > tol) && (i < maxIter)
    i = i + 1;

    % *** PASTE YOUR METHOD-SPECIFIC FORMULA HERE ***
    % Different for each method!

    % Compute error
    if i == 1
        err = inf; % First iteration has no previous value
    else
        err = abs((x_new - x_old) / x_new) * 100; % Relative error
    end

    % Print iteration results
    fprintf('%-5d %-15.8f %-15.8f %-15.6f\n', i, x_new, f(x_new), err);

    % Update for next iteration
    x_old = x_new;
    % x0 = x_new; (uncomment based on method)
end

%-----
% STEP 7: DISPLAY FINAL RESULT
%-----

fprintf('%s\n', '=====');
fprintf('\n✅ Approximate root = %.8f\n', x_new);
fprintf('✅ Found in %d iterations\n', i);
fprintf('✅ Tolerance = %.1e\n', tol);
fprintf('✅ Final error = %.6e\n\n', err);

---

## 🔍 METHOD-SPECIFIC FORMULAS (The ONLY thing that changes!)

### **1. BISECTION METHOD**
- **Category:** Closed/Bracket method
- **Requires:** Function only
- **Formula in Loop:**
matlab
Xm = (a + b) / 2; % Midpoint

if f(a) * f(Xm) < 0
    b = Xm; % Root in [a, Xm]
else
    a = Xm; % Root in [Xm, b]
end

x_new = Xm;

---

### **2. NEWTON-RAPHSON METHOD**
- **Category:** Open method
- **Requires:** Function + First derivative
- **Formula in Loop:**
matlab
f0 = f(x0);
f1 = df(x0);

x_new = x0 - f0 / f1; % Newton's formula

x0 = x_new;

---

### **3. SECANT METHOD**
- **Category:** Open method
- **Requires:** Function only (no derivative!)
- **Requires:** TWO initial guesses (x0, x1)
- **Formula in Loop:**
matlab

```

```

x2 = x1 - f(x1) * (x1 - x0) / (f(x1) - f(x0));

x0 = x1;
x1 = x2;
x_new = x2;

---

### **4. REGULA FALSI METHOD (False Position)**
- **Category:** Closed/Bracket method
- **Requires:** Function only
- **Formula in Loop:**
matlab
x_new = (a*f(b) - b*f(a)) / (f(b) - f(a)); % Linear interpolation

if f(a) * f(x_new) < 0
    b = x_new; % Root in [a, x_new]
else
    a = x_new; % Root in [x_new, b]
end

---

### **5. FIXED POINT ITERATION**
- **Category:** Open method
- **Requires:** Rearranged function g(x) where x = g(x)
- **Condition:** |g'(x)| < 1 at root for convergence
- **Formula in Loop:**
matlab
x_new = g(x0); % Simple iteration formula

x0 = x_new;

---

### **6. MODIFIED NEWTON-RAPHSON METHOD**
- **Category:** Open method
- **Requires:** Function + First & Second derivatives
- **Used for:** Multiple roots (when f(x) = 0 AND f'(x) = 0)
- **Formula in Loop (Variant 1):**
matlab
f0 = f(x0);
f1 = df(x0);
f2 = d2f(x0);

x_new = x0 - (f0 * f1) / (f1^2 - f0 * f2);

x0 = x_new;

---

```

📊 QUICK REFERENCE TABLE

Method	Type	Input	Convergence	Best For
Bisection	Closed	f(x), [a,b]	Slow, Guaranteed	Bracketing
Newton-Raphson	Open	f(x), f'(x), x0	Fast, Quadratic	Simple roots
Secant	Open	f(x), x0, x1	Fast	No derivative
Regula Falsi	Closed	f(x), [a,b]	Medium	Bracketing
Fixed Point	Open	g(x), x0	Variable	Rearranged form
Modified NR	Open	f(x), f'(x), f''(x), x0	Very Fast	Multiple roots

💡 KEY CONCEPTS TO REMEMBER

```

### **Error Calculations**
matlab
% Absolute Error
err_abs = abs(x_new - x_old);

% Relative Error (percentage)
err_rel = abs((x_new - x_old) / x_new) * 100;

### **Stopping Criteria**

```

```

matlab
% Stop when:
while (err > tol) && (i < maxIter)
    % - Error is less than tolerance, OR
    % - Maximum iterations reached
end

```

```

### **Convergence Checks**
matlab
% Bracket methods (Bisection, Regula Falsi):
if f(a) * f(b) > 0
    % Same sign = no root in interval!
end

% Fixed Point:
if abs(dg_dx) >= 1
    % Won't converge!
end

```

🧠 COMMON MISTAKES TO AVOID

1. ❌ ****Forget clc; clear; close all;**** → Always clean workspace
2. ❌ ****Wrong error formula**** → Use relative error: $\text{abs}((\text{new} - \text{old})/\text{new}) * 100$
3. ❌ ****Check bracket validity**** → $f(a) \cdot f(b) < 0$ MUST be true
4. ❌ ****Update variables wrongly**** → Track which variable gets updated
5. ❌ ****Print header before loop**** → Makes table readable
6. ❌ ****Forget to initialize err = inf**** → So first iteration doesn't trigger early stop
7. ❌ ****Wrong derivative**** → Double-check your calculus!

📝 STEP-BY-STEP WRITING STRATEGY FOR QUIZ

****When writing by hand, follow this order:****

1. ****Write header section**** (clc, clear, close all)
2. ****Define function**** (f = @(x) ...)
3. ****Get input**** (x0 = input(...))
4. ****Set parameters**** (tol, maxIter, i=0, err=inf)
5. ****Print header**** (fprintf for table)
6. ****Write while loop condition**** (err > tol && i < maxIter)
7. ****Update iteration counter**** (i = i + 1)
8. ****Write method formula**** (THE KEY PART!)
9. ****Calculate error**** (err = $\text{abs}((\text{new}-\text{old})/\text{new}) * 100$)
10. ****Print results**** (fprintf)
11. ****Update variables**** (x0 = x_new, etc)
12. ****Close loop and print final result****

🎯 YOUR ACTUAL CODE ANALYSIS

Your codes follow this structure:

- ✓ Proper initialization
- ✓ Good error calculations
- ✓ Clear output formatting
- ✓ Proper stopping criteria

Standardize by using the template above!

📚 PRACTICE PROBLEMS

****Bisection Method****
Find root of $f(x) = x^3 - 2x - 5$ in interval [2, 3]

****Newton-Raphson****
Find root of $f(x) = e^x - 5x$ with $x_0 = 1$

****Secant Method****
Find root of $f(x) = \cos(x)$ with $x_0 = 0, x_1 = 1$

****Regula Falsi****
Find root of $f(x) = x^3 - 2x - 5$ in interval [2, 3]

Fixed Point

Rearrange $x^3 + x - 1 = 0$ to $x = 1/(x^2 + 1)$ and iterate from $x_0 = 0.5$

✨ FINAL TIPS FOR QUIZ

1. **Use the template!** Write it down first
2. **Identify the method clearly** (what's being asked)
3. **Choose correct formula** from the table
4. **Show error calculation** step-by-step
5. **Keep formatting consistent** for easy grading
6. **Test your code mentally** before submitting
7. **Label all outputs** clearly

Good luck! 🍀 You've got this!

"

ChatGPT said:

🎓 MATLAB Numerical Methods Quiz Preparation Guide

Quiz Date: October 20, 2025

Topics: Root Finding Methods (Bisection, Newton-Raphson, Secant, Regula Falsi, Fixed Point, Modified Newton)

📄 UNIVERSAL MATLAB TEMPLATE FOR ALL ROOT-FINDING METHODS

USE THIS EXACT PATTERN - Change only the formulas and method names!

```
%=====
% METHOD NAME: [Method Name Here]
%=====

clc
clear
close all

%=====
% STEP 1: DEFINE FUNCTION AND DERIVATIVES (If needed)
%=====

f = @(x) [FUNCTION HERE];          % Main function

% Uncomment derivative if method requires it
% df = @(x) [DERIVATIVE HERE];     % First derivative (Newton, Modified Newton)
% d2f = @(x) [2ND DERIVATIVE HERE]; % Second derivative (Modified Newton)

%=====
% STEP 2: INPUT INITIAL GUESS(ES)
%=====

% For Bisection/Regula Falsi (bracket methods):
% a = input('Enter value of a: ');
% b = input('Enter value of b: ');

% For Newton/Secant/Fixed Point (open methods):
x0 = input('Enter initial guess x0: ');

%=====
% STEP 3: VALIDATE INPUT (For bracket methods)
%=====

% Uncomment for bracket methods:
% while f(a) * f(b) > 0
%     disp('f(a) and f(b) must have opposite signs!');
%     a = input('Enter new value of a: ');
%     b = input('Enter new value of b: ');
% end

%=====
% STEP 4: SET PARAMETERS
%=====

tol = 1e-6;          % Tolerance (convergence criterion)
maxIter = 50;        % Maximum iterations
i = 0;               % Iteration counter
err = inf;           % Initialize error

%=====
% STEP 5: PRINT HEADER
%=====
```

```

fprintf('\n%s\n', '=====');
fprintf('%-5s %-15s %-15s %-15s %-15s\n', ...
    'Iter', 'x_value', 'f(x)', 'Error(%)', 'Other');
fprintf('%s\n', '=====');

%-----
% STEP 6: ITERATION LOOP
%-----

while (err > tol) && (i < maxIter)
    i = i + 1;

    % *** PASTE YOUR METHOD-SPECIFIC FORMULA HERE ***
    % Different for each method!

    % Compute error
    if i == 1
        err = inf; % First iteration has no previous value
    else
        err = abs((x_new - x_old) / x_new) * 100; % Relative error
    end

    % Print iteration results
    fprintf('%-5d %-15.8f %-15.8f %-15.6f\n', i, x_new, f(x_new), err);

    % Update for next iteration
    x_old = x_new;
    % x0 = x_new; (uncomment based on method)
end

%-----
% STEP 7: DISPLAY FINAL RESULT
%-----

fprintf('%s\n', '=====');
fprintf('\n✅ Approximate root = %.8f\n', x_new);
fprintf('✅ Found in %d iterations\n', i);
fprintf('✅ Tolerance = %.1e\n', tol);
fprintf('✅ Final error = %.6e\n\n', err);

```

METHOD-SPECIFIC FORMULAS (The ONLY thing that changes!)

1. BISECTION METHOD

Category: Closed/Bracket method

Requires: Function only

Formula in Loop:

```

Xm = (a + b) / 2; % Midpoint

if f(a) * f(Xm) < 0
    b = Xm; % Root in [a, Xm]
else
    a = Xm; % Root in [Xm, b]
end

```

x_new = Xm;

2. NEWTON-RAPHSON METHOD

Category: Open method

Requires: Function + First derivative

Formula in Loop:

```

f0 = f(x0);
f1 = df(x0);

x_new = x0 - f0 / f1; % Newton's formula

```

x0 = x_new;

3. SECANT METHOD

Category: Open method

Requires: Function only (no derivative!)

Requires: TWO initial guesses (x0, x1)

Formula in Loop:

```
x2 = x1 - f(x1) * (x1 - x0) / (f(x1) - f(x0));
```

```
x0 = x1;
```

```
x1 = x2;
```

```
x_new = x2;
```

4. REGULA FALSI METHOD (False Position)

Category: Closed/Bracket method

Requires: Function only

Formula in Loop:

```
x_new = (a*f(b) - b*f(a)) / (f(b) - f(a)); % Linear interpolation
```

```
if f(a) * f(x_new) < 0
```

```
    b = x_new; % Root in [a, x_new]
```

```
else
```

```
    a = x_new; % Root in [x_new, b]
```

```
end
```

5. FIXED POINT ITERATION

Category: Open method

Requires: Rearranged function $g(x)$ where $x = g(x)$

Condition: $|g'(x)| < 1$ at root for convergence

Formula in Loop:

```
x_new = g(x0); % Simple iteration formula
```

```
x0 = x_new;
```

6. MODIFIED NEWTON-RAPHSON METHOD

Category: Open method

Requires: Function + First & Second derivatives

Used for: Multiple roots (when $f(x) = 0$ AND $f'(x) = 0$)

Formula in Loop (Variant 1):

```
f0 = f(x0);
```

```
f1 = df(x0);
```

```
f2 = d2f(x0);
```

```
x_new = x0 - (f0 * f1) / (f1^2 - f0 * f2);
```

```
x0 = x_new;
```

 QUICK REFERENCE TABLE

Method	Type	Input	Convergence	Best For
Bisection	Closed	$f(x)$, $[a, b]$	Slow, Guaranteed	Bracketing
Newton-Raphson	Open	$f(x)$, $f'(x)$, x_0	Fast, Quadratic	Simple roots
Secant	Open	$f(x)$, x_0 , x_1	Fast	No derivative
Regula Falsi	Closed	$f(x)$, $[a, b]$	Medium	Bracketing
Fixed Point	Open	$g(x)$, x_0	Variable	Rearranged form
Modified NR	Open	$f(x)$, $f'(x)$, $f''(x)$, x_0	Very Fast	Multiple roots

 KEY CONCEPTS TO REMEMBER

Error Calculations

% Absolute Error

```
err_abs = abs(x_new - x_old);
```

% Relative Error (percentage)

```
err_rel = abs((x_new - x_old) / x_new) * 100;
```

Stopping Criteria

% Stop when:

```
while (err > tol) && (i < maxIter)
```

```
    % - Error is less than tolerance, OR
```

```
    % - Maximum iterations reached
```

```
end
```

Convergence Checks

% Bracket methods (Bisection, Regula Falsi):

```
if f(a) * f(b) > 0
```

```
    % Same sign = no root in interval!
```

```
end
```

% Fixed Point:

```
if abs(dg_dx) >= 1
```

```
    % Won't converge!
```

```
end
```

 COMMON MISTAKES TO AVOID

 Forget `clc`; `clear`; `close all`; → Always clean workspace

- ✗ Wrong error formula → Use relative error: $\text{abs}((\text{new} - \text{old})/\text{new}) * 100$
- ✗ Check bracket validity → $f(a) \cdot f(b) < 0$ MUST be true
- ✗ Update variables wrongly → Track which variable gets updated
- ✗ Print header before loop → Makes table readable
- ✗ Forget to initialize $\text{err} = \text{inf}$ → So first iteration doesn't trigger early stop
- ✗ Wrong derivative → Double-check your calculus!

 **STEP-BY-STEP WRITING STRATEGY FOR QUIZ**
When writing by hand, follow this order:

Write header section (clc, clear, close all)

Define function ($f = @(x) \dots$)

Get input ($x_0 = \text{input}(\dots)$)

Set parameters (tol, maxIter, $i=0$, $\text{err}=\text{inf}$)

Print header (fprintf for table)

Write while loop condition ($\text{err} > \text{tol} \ \&\& \ i < \text{maxIter}$)

Update iteration counter ($i = i + 1$)

Write method formula (THE KEY PART!)

Calculate error ($\text{err} = \text{abs}((\text{new}-\text{old})/\text{new})*100$)

Print results (fprintf)

Update variables ($x_0 = x_{\text{new}}$, etc)

Close loop and print final result

 **YOUR ACTUAL CODE ANALYSIS**
Your codes follow this structure:

- ✓ Proper initialization
- ✓ Good error calculations
- ✓ Clear output formatting
- ✓ Proper stopping criteria

Standardize by using the template above!

 **PRACTICE PROBLEMS**

Bisection Method

Find root of $f(x) = x^3 - 2x - 5$ in interval $[2, 3]$

Newton-Raphson

Find root of $f(x) = e^x - 5x$ with $x_0 = 1$

Secant Method

Find root of $f(x) = \cos(x)$ with $x_0 = 0$, $x_1 = 1$

Regula Falsi

Find root of $f(x) = x^3 - 2x - 5$ in interval $[2, 3]$

Fixed Point

Rearrange $x^3 + x - 1 = 0$ to $x = 1/(x^2 + 1)$ and iterate from $x_0 = 0.5$

✨ **FINAL TIPS FOR QUIZ**

Use the template! Write it down first

Identify the method clearly (what's being asked)

Choose correct formula from the table

Show error calculation step-by-step

Keep formatting consistent for easy grading

Test your code mentally before submitting

Label all outputs clearly

Good luck! 🍀 You've got this!

You said:
give formula in correct appearance "Bisection Method

Find root of $f(x) = x^3 - 2x - 5$ in interval $[2, 3]$

Newton-Raphson

Find root of $f(x) = e^x - 5x$ with $x_0 = 1$

Secant Method

Find root of $f(x) = \cos(x)$ with $x_0 = 0, x_1 = 1$

Regula Falsi

Find root of $f(x) = x^3 - 2x - 5$ in interval $[2, 3]$

Fixed Point

Rearrange $x^3 + x - 1 = 0$ to $x = 1/(x^2 + 1)$ and iterate from $x_0 = 0.5$ "

ChatGPT said:

Here's your content rewritten in Word/LibreOffice-friendly math format (so it appears properly formatted when pasted – all exponents, fractions, and symbols display correctly):

Bisection Method

Find the root of

$$f(x) = x^3 - 2x - 5$$

in the interval $[2, 3]$

Newton-Raphson Method

Find the root of

$$f(x) = e^x - 5x$$

with initial guess

$$x_0 = 1$$

Secant Method
Find the root of

$$f(x) = \cos(x)$$

(
x
)
 $f(x)=\cos(x)$
with initial guesses

x
0
=
0
,
x
1
=
1
x
0

=0, x
1

=1
Regula Falsi Method (False Position)
Find the root of

f
(
x
)
=
x
3
-
2
x
-
5
 $f(x)=x$
3
-2x-5
in the interval [2, 3]

Fixed Point Iteration
Rearrange the equation

x
3
+
x
-
1
=
0
x
3
+x-1=0
into the form

x
=
1
x
2
+
1
x=
x
2
+1
1

and iterate starting from

x
0
=
0.5
x

0

=0.5

✓ Tip for Word or LibreOffice Writer:

Go to Insert → Object → Formula (or press Alt + = in Word).

Then paste each formula line (between \[and \]) inside the formula editor.
They will appear beautifully formatted as mathematical expressions.

You said:

MATLAB ROOT-FINDING METHODS - QUICK REFERENCE SHEET Print & Keep with you for the quiz!
--

1. BISECTION METHOD

PURPOSE: Find root of $f(x) = 0$ in interval $[a, b]$
REQUIRES: $f(x)$, interval $[a, b]$ where $f(a) \cdot f(b) < 0$
TYPE: Closed method (bracket method)
SPEED: SLOW but guaranteed
CONVERGENCE: If f continuous and $f(a) \cdot f(b) < 0 \rightarrow$ ALWAYS converges

ITERATION FORMULA:
$$X_m = (a + b) / 2$$

CONDITION CHECK:

```
if f(a) * f(Xm) < 0
    b = Xm      (root in [a, Xm])
else
    a = Xm      (root in [Xm, b])
end
```

KEY CODE TEMPLATE:

```
while (err > tol) && (i < maxIter)
    i = i + 1;
    Xm = (a + b) / 2;
    if f(a) * f(Xm) < 0
        b = Xm;
    else
        a = Xm;
    end
    err = abs((Xm - Xm_old) / Xm) * 100;
    Xm_old = Xm;
end
```

WHEN TO USE: When you have two points bracketing the root

2. NEWTON-RAPHSON METHOD

PURPOSE: Find root of $f(x) = 0$
REQUIRES: $f(x)$, $f'(x)$, initial guess x_0
TYPE: Open method
SPEED: FAST (quadratic convergence)
CONVERGENCE: Fast near root, may diverge far from root

ITERATION FORMULA:
$$x_{\text{new}} = x_0 - f(x_0) / f'(x_0)$$

KEY CODE TEMPLATE:

```
while (err > tol) && (i < maxIter)
    i = i + 1;
    f0 = f(x0);
    f1 = df(x0);
    x_new = x0 - f0 / f1;
    err = abs((x_new - x0) / x_new) * 100;
    x0 = x_new;
end
```

WHEN TO USE: When you have $f'(x)$ and good initial guess

3. SECANT METHOD

PURPOSE: Find root of $f(x) = 0$ WITHOUT derivative
REQUIRES: $f(x)$, TWO initial guesses (x_0 , x_1)
TYPE: Open method
SPEED: FAST (superlinear, ~ 1.618)
CONVERGENCE: Very good, faster than bisection

ITERATION FORMULA:
$$x_{\text{new}} = x_1 - f(x_1) \cdot (x_1 - x_0) / (f(x_1) - f(x_0))$$

KEY CODE TEMPLATE:

```
| while (err > tol) && (i < maxIter)
|     i = i + 1;
|     x2 = x1 - f(x1) * (x1 - x0) / (f(x1) - f(x0));
|     err = abs((x2 - x1) / x2) * 100;
|     x0 = x1;
|     x1 = x2;
| end
```

WHEN TO USE: When you can't compute derivative easily

4. REGULA FALSI (False Position Method)

PURPOSE: Find root using linear interpolation on bracket
REQUIRES: $f(x)$, interval $[a, b]$ where $f(a) \cdot f(b) < 0$
TYPE: Closed method (bracket method)
SPEED: MEDIUM (better than bisection)
CONVERGENCE: Guaranteed if f continuous and $f(a) \cdot f(b) < 0$

ITERATION FORMULA:
$$x_{\text{new}} = (a \cdot f(b) - b \cdot f(a)) / (f(b) - f(a))$$

CONDITION CHECK:

```
| if f(a) * f(x_new) < 0
|     b = x_new
| else
|     a = x_new
| end
```

KEY CODE TEMPLATE:

```
| while (err > tol) && (i < maxIter)
|     i = i + 1;
|     x_new = (a*f(b) - b*f(a)) / (f(b) - f(a));
|     if f(a) * f(x_new) < 0
|         b = x_new;
|     else
|         a = x_new;
|     end
|     err = abs((x_new - x_old) / x_new) * 100;
|     x_old = x_new;
| end
```

WHEN TO USE: When bisection is too slow but you need guarantee

5. FIXED POINT ITERATION

PURPOSE: Find fixed point $x = g(x)$ by iteration
REQUIRES: Rearranged $g(x)$, initial guess x_0
TYPE: Open method
SPEED: VARIABLE (depends on $g'(x)$)
CONVERGENCE: Need $|g'(x)| < 1$ near root

ITERATION FORMULA:
$$x_{\text{new}} = g(x_0)$$

where $g(x)$ is the rearranged form: $x = g(x)$

REARRANGEMENT EXAMPLES:
 $f(x) = 0 \rightarrow x = g(x)$
 $x^3 + x - 1 = 0 \rightarrow x = \sqrt[3]{1 - x}$
 $2x - \log(x) - 7 = 0 \rightarrow x = \exp(2x - 7)$

KEY CODE TEMPLATE:

```
| while (err > tol) && (i < maxIter)
```

```

    i = i + 1;
    x_new = g(x0);
    err = abs((x_new - x0) / x_new) * 100;
    x0 = x_new;
end

```

CONVERGENCE CHECK:

```

dg = abs(dg_dx at root)
if dg < 1 → CONVERGES
if dg > 1 → DIVERGES

```

WHEN TO USE: When equation is easily rearrangeable

6. MODIFIED NEWTON-RAPHSON METHOD

PURPOSE: Find MULTIPLE roots where $f(x) = f'(x) = 0$
 REQUIRES: $f(x)$, $f'(x)$, $f''(x)$, initial guess x_0
 TYPE: Open method
 SPEED: VERY FAST (superlinear for multiple roots)
 CONVERGENCE: For multiple roots: EXCELLENT

ITERATION FORMULA:

$$x_{\text{new}} = x_0 - (f(x_0) \cdot f'(x_0)) / ([f'(x_0)]^2 - f(x_0) \cdot f''(x_0))$$

KEY CODE TEMPLATE:

```

while (err > tol) && (i < maxIter)
    i = i + 1;
    f0 = f(x0);
    f1 = df(x0);
    f2 = d2f(x0);
    x_new = x0 - (f0 * f1) / (f1^2 - f0 * f2);
    err = abs((x_new - x0) / x_new) * 100;
    x0 = x_new;
end

```

WHEN TO USE: When root has multiplicity > 1 (tangent to x-axis)

UNIVERSAL PATTERN - ALL METHODS USE THIS STRUCTURE

```

STEP 1: clc; clear; close all;
STEP 2: f = @(x) [function];
STEP 3: [Get input: a, b OR x0]
STEP 4: [Validate if bracket method]
STEP 5: tol = 1e-6; maxIter = 50; i = 0; err = inf;
STEP 6: fprintf(['Print header table']);
STEP 7: while (err > tol) && (i < maxIter)
    i = i + 1;
    [FORMULA HERE - changes per method]
    [Error calculation]
    [fprintf results]
    [Update variables]
end
STEP 8: fprintf(['Final results']);

```

COMMON ERROR FORMULAS

Relative Error (MOST COMMON):

$$\text{err} = \text{abs}((x_{\text{new}} - x_{\text{old}}) / x_{\text{new}}) * 100 \quad [\%]$$

Absolute Error:

$$\text{err} = \text{abs}(x_{\text{new}} - x_{\text{old}})$$

First Iteration:

$$\text{err} = \text{inf} \quad (\text{no previous value to compare})$$

STOPPING CRITERIA

Stop when ANY of these is true:

1. err < tol (error tolerance reached)
2. i > maxIter (maximum iterations exceeded)
3. f(x_new) = 0 (exact root found - optional)
4. Denominator = 0 (mathematical error - avoid!)

FUNCTION DEFINITIONS IN MATLAB

```
f = @(x) x^2 - 12;           % Parabola
f = @(x) exp(x) - 5*x;       % Exponential
f = @(x) cos(x);             % Trigonometric
f = @(x) x^3 - 2*x - 5;      % Cubic

Derivatives:
df = @(x) 2*x;               % d/dx(x^2)
df = @(x) exp(x) - 5;        % d/dx(exp(x) - 5x)
df = @(x) -sin(x);           % d/dx(cos(x))
df = @(x) 3*x^2 - 2;         % d/dx(x^3 - 2x)
```

IMPORTANT NOTES FOR QUIZ

- ✓ Always start with: clc; clear; close all;
- ✓ Use descriptive variable names (x0, x_new, err, etc)
- ✓ Print header BEFORE the loop (makes output readable)
- ✓ Initialize: i = 0, err = inf BEFORE loop
- ✓ Use RELATIVE error by default: abs((new-old)/new)*100
- ✓ For bracket methods: ALWAYS check f(a)*f(b) < 0
- ✓ For fixed point: Rearrange equation FIRST, then code
- ✓ Use fprintf() with format strings for clean output
- ✓ Update variables correctly at end of each iteration
- ✓ Keep same pattern for all methods - ONLY formula changes!

ChatGPT said:

Here's your Root-Finding Methods Quick Reference Sheet in a perfectly formatted version that you can copy directly into Word or LibreOffice Writer – the lines, boxes, and formulas will stay aligned and display cleanly in monospace or code-style font (like Consolas or Courier New).

MATLAB ROOT-FINDING METHODS - QUICK REFERENCE SHEET Print & Keep with you for the quiz!

1. BISECTION METHOD

PURPOSE: Find root of $f(x) = 0$ in interval $[a, b]$
REQUIRES: $f(x)$, interval $[a, b]$ where $f(a) \cdot f(b) < 0$
TYPE: Closed method (bracket method)
SPEED: SLOW but guaranteed
CONVERGENCE: If f continuous and $f(a) \cdot f(b) < 0 \rightarrow$ ALWAYS converges

ITERATION FORMULA:
$$X_m = (a + b) / 2$$

CONDITION CHECK:

```
if f(a) * f(Xm) < 0
    b = Xm      (root in [a, Xm])
else
    a = Xm      (root in [Xm, b])
end
```

KEY CODE TEMPLATE:

```
while (err > tol) && (i < maxIter)
    i = i + 1;
    Xm = (a + b) / 2;
    if f(a) * f(Xm) < 0
        b = Xm;
    else
        a = Xm;
    end
    err = abs((Xm - Xm_old) / Xm) * 100;
    Xm_old = Xm;
end
```

WHEN TO USE: When you have two points bracketing the root

2. NEWTON-RAPHSON METHOD

PURPOSE: Find root of $f(x) = 0$
REQUIRES: $f(x)$, $f'(x)$, initial guess x_0
TYPE: Open method
SPEED: FAST (quadratic convergence)
CONVERGENCE: Fast near root, may diverge far from root

ITERATION FORMULA:
$$x_{\text{new}} = x_0 - f(x_0) / f'(x_0)$$

KEY CODE TEMPLATE:

```
while (err > tol) && (i < maxIter)
    i = i + 1;
    f0 = f(x0);
    f1 = df(x0);
    x_new = x0 - f0 / f1;
    err = abs((x_new - x0) / x_new) * 100;
    x0 = x_new;
end
```

WHEN TO USE: When you have $f'(x)$ and good initial guess

3. SECANT METHOD

PURPOSE: Find root of $f(x) = 0$ WITHOUT derivative
REQUIRES: $f(x)$, TWO initial guesses (x_0 , x_1)
TYPE: Open method
SPEED: FAST (superlinear, ~ 1.618)
CONVERGENCE: Very good, faster than bisection

ITERATION FORMULA:
$$x_{\text{new}} = x_1 - f(x_1) \cdot (x_1 - x_0) / (f(x_1) - f(x_0))$$

KEY CODE TEMPLATE:

```
while (err > tol) && (i < maxIter)
    i = i + 1;
    x2 = x1 - f(x1) * (x1 - x0) / (f(x1) - f(x0));
    err = abs((x2 - x1) / x2) * 100;
    x0 = x1;
    x1 = x2;
end
```

WHEN TO USE: When you can't compute derivative easily

4. REGULA FALSI (False Position Method)

PURPOSE: Find root using linear interpolation on bracket
REQUIRES: $f(x)$, interval $[a, b]$ where $f(a) \cdot f(b) < 0$
TYPE: Closed method (bracket method)
SPEED: MEDIUM (better than bisection)
CONVERGENCE: Guaranteed if f continuous and $f(a) \cdot f(b) < 0$

ITERATION FORMULA:
$$x_{\text{new}} = (a \cdot f(b) - b \cdot f(a)) / (f(b) - f(a))$$

CONDITION CHECK:

```
if f(a) * f(x_new) < 0
    b = x_new
else
    a = x_new
end
```

KEY CODE TEMPLATE:

```
while (err > tol) && (i < maxIter)
    i = i + 1;
    x_new = (a*f(b) - b*f(a)) / (f(b) - f(a));
    if f(a) * f(x_new) < 0
```

```

        b = x_new;
    else
        a = x_new;
    end
    err = abs((x_new - x_old) / x_new) * 100;
    x_old = x_new;
end

```

WHEN TO USE: When bisection is too slow but you need guarantee

5. FIXED POINT ITERATION

PURPOSE: Find fixed point $x = g(x)$ by iteration
REQUIRES: Rearranged $g(x)$, initial guess x_0
TYPE: Open method
SPEED: VARIABLE (depends on $g'(x)$)
CONVERGENCE: Need $|g'(x)| < 1$ near root

ITERATION FORMULA:
 $x_{\text{new}} = g(x_0)$
 where $g(x)$ is the rearranged form: $x = g(x)$

REARRANGEMENT EXAMPLES:
 $f(x) = 0 \rightarrow x = g(x)$
 $x^3 + x - 1 = 0 \rightarrow x = \sqrt[3]{1 - x}$
 $2x - \log(x) - 7 = 0 \rightarrow x = \exp(2x - 7)$

KEY CODE TEMPLATE:

```

while (err > tol) && (i < maxIter)
    i = i + 1;
    x_new = g(x0);
    err = abs((x_new - x0) / x_new) * 100;
    x0 = x_new;
end

```

CONVERGENCE CHECK:
 $dg = \text{abs}(dg_dx \text{ at root})$
 if $dg < 1 \rightarrow$ CONVERGES
 if $dg > 1 \rightarrow$ DIVERGES

WHEN TO USE: When equation is easily rearrangeable

6. MODIFIED NEWTON-RAPHSON METHOD

PURPOSE: Find MULTIPLE roots where $f(x) = f'(x) = 0$
REQUIRES: $f(x)$, $f'(x)$, $f''(x)$, initial guess x_0
TYPE: Open method
SPEED: VERY FAST (superlinear for multiple roots)
CONVERGENCE: For multiple roots: EXCELLENT

ITERATION FORMULA:
 $x_{\text{new}} = x_0 - (f(x_0) \cdot f'(x_0)) / ([f'(x_0)]^2 - f(x_0) \cdot f''(x_0))$

KEY CODE TEMPLATE:

```

while (err > tol) && (i < maxIter)
    i = i + 1;
    f0 = f(x0);
    f1 = df(x0);
    f2 = d2f(x0);
    x_new = x0 - (f0 * f1) / (f1^2 - f0 * f2);
    err = abs((x_new - x0) / x_new) * 100;
    x0 = x_new;
end

```

WHEN TO USE: When root has multiplicity > 1 (tangent to x-axis)

UNIVERSAL PATTERN - ALL METHODS USE THIS STRUCTURE

STEP 1: `clc; clear; close all;`
 STEP 2: `f = @(x) [function];`
 STEP 3: `[Get input: a, b OR x0]`

```

STEP 4: [Validate if bracket method]
STEP 5: tol = 1e-6; maxIter = 50; i = 0; err = inf;
STEP 6: fprintf(['Print header table']);
STEP 7: while (err > tol) && (i < maxIter)
        i = i + 1;
        [FORMULA HERE - changes per method]
        [Error calculation]
        [fprintf results]
        [Update variables]
    end
STEP 8: fprintf(['Final results']);

```

COMMON ERROR FORMULAS

Relative Error (MOST COMMON):

```
err = abs((x_new - x_old) / x_new) * 100 [%]
```

Absolute Error:

```
err = abs(x_new - x_old)
```

First Iteration:

```
err = inf (no previous value to compare)
```

STOPPING CRITERIA

Stop when ANY of these is true:

1. $err < tol$ (error tolerance reached)
2. $i > maxIter$ (maximum iterations exceeded)
3. $f(x_{new}) = 0$ (exact root found - optional)
4. Denominator = 0 (mathematical error - avoid!)

FUNCTION DEFINITIONS IN MATLAB

```

f = @(x) x^2 - 12;           % Parabola
f = @(x) exp(x) - 5*x;       % Exponential
f = @(x) cos(x);             % Trigonometric
f = @(x) x^3 - 2*x - 5;      % Cubic

```

Derivatives:

```

df = @(x) 2*x;               % d/dx(x^2)
df = @(x) exp(x) - 5;        % d/dx(exp(x) - 5x)
df = @(x) -sin(x);           % d/dx(cos(x))
df = @(x) 3*x^2 - 2;         % d/dx(x^3 - 2x)

```

IMPORTANT NOTES FOR QUIZ

- ✓ Always start with: `clc; clear; close all;`
- ✓ Use descriptive variable names (`x0`, `x_new`, `err`, etc)
- ✓ Print header BEFORE the loop (makes output readable)
- ✓ Initialize: `i = 0`, `err = inf` BEFORE loop
- ✓ Use RELATIVE error by default: `abs((new-old)/new)*100`
- ✓ For bracket methods: ALWAYS check $f(a)*f(b) < 0$
- ✓ For fixed point: Rearrange equation FIRST, then code
- ✓ Use `fprintf()` with format strings for clean output
- ✓ Update variables correctly at end of each iteration
- ✓ Keep same pattern for all methods - ONLY formula changes!
- ✓ ☒ Best copy method:

Use `Ctrl + A` → `Ctrl + C` from above code block.

In Word/LibreOffice Writer, use "Consolas" or "Courier New" font, 10-11 pt size.

Then choose "Keep Text Only" when pasting.

This preserves all box lines and alignment perfectly.

No file chosenNo file chosen
ChatGPT can make mistakes. Check important info.

Save as PDF
▼

Save as PDF